

Relatório do Trabalho Prático

Processamento de Linguagens

Implementação de Analisador e Interpretador para a Linguagem LFun

LESI - 2º Ano, 2º Semestre

8 de maio de 2026

Conteúdo

1	Introdução	3
2	Objectivos	4
3	Estrutura do Projeto	5
3.1	Organização Geral	5
3.2	Arquitectura do Sistema	5
3.2.1	Componentes Principais	6
4	Implementação	8
4.1	Parte A - Análise Léxica	8
4.1.1	Funcionalidades	8
4.1.2	Estrutura do Lexer	8
4.2	Parte B - Análise Sintática	9
4.2.1	Gramática	9
4.2.2	Utilização de PLY	9
4.3	Parte C - Avaliação Básica	10
4.3.1	Características	10
4.4	Parte D - Representação Intermédia e Pattern Matching	10
4.4.1	Nós da AST	10
4.4.2	Pattern Matching	11
4.4.3	Exemplos de Pattern Matching	11
4.4.4	Avaliação Genérica	11
5	Tecnologias Utilizadas	12
6	Testes e Validação	13
6.1	Estratégia de Testes	13
6.1.1	Testes Unitários	13
6.1.2	Cobertura de Testes	13
6.2	Resultados de Teste	14
7	Resultados e Demonstração	15
7.1	Interface REPL	15
7.2	Exemplo Completo	15
7.3	Conformidade com Requisitos	16
8	Conclusão	17
8.1	Aprendizagens Principais	17
8.2	Funcionalidades Implementadas	17
8.3	Possíveis Melhorias Futuras	17
8.4	Reflexão Final	18

A	Estrutura de Ficheiros	19
B	Repositório GitHub	20

Capítulo 1

Introdução

Este relatório documenta o trabalho prático da unidade curricular de **Processamento de Linguagens**, desenvolvido no contexto da Licenciatura em Engenharia de Sistemas Informáticos (LESI), 2º ano, 2º semestre.

O trabalho centrou-se no desenvolvimento progressivo de um analisador completo para uma linguagem funcional denominada **LFun** (Language Functional), incluindo componentes de análise léxica, análise sintática, avaliação básica e, finalmente, um sistema robusto com representação intermédia e suporte avançado a padrões.

O projeto foi estruturado em quatro fases incrementais (A, B, C e D), permitindo uma construção gradual e consolidação de conceitos fundamentais no processamento de linguagens.

Capítulo 2

Objectivos

O presente trabalho teve como objectivos principais:

1. Compreender e aplicar os conceitos teóricos de processamento de linguagens, nomeadamente:
 - Análise léxica e tokenização
 - Análise sintática e gramáticas formais
 - Representação intermédia (IR)
 - Avaliação e interpretação de código
2. Implementar um analisador léxico capaz de:
 - Identificar palavras-chave, operadores e literais
 - Processar comentários de linha e bloco
 - Gerar tokens correctamente
3. Desenvolver um analisador sintático que:
 - Verifique a conformidade estrutural do código
 - Valide expressões complexas
 - Produza uma representação sintáctica válida
4. Criar um interpretador com funcionalidades avançadas:
 - Avaliação de expressões
 - Suporte a pattern matching robusto
 - Representação intermédia genérica (AST)
5. Garantir qualidade através de testes abrangentes

Capítulo 3

Estrutura do Projeto

3.1 Organização Geral

O projeto foi desenvolvido em quatro etapas, cada uma construindo sobre a anterior:

Parte	Objectivo	Componentes
A	Análise Léxica	lexer.py, lexer_test
B	Análise Sintática	lexer.py, grammar.py, parser
C	Avaliação Básica	parser.py, main.py, test
D	IR e Pattern Matching	ast_nodes.py, eval.py, test_eval.py

3.2 Arquitectura do Sistema

O sistema segue uma arquitetura em camadas:

```
1
2      C ó d i g o   F o n t e   ( E n t r a d a )
3
4
5      A N   L I S E   L   X I C A
6      ( L e x e r )
7      l e x e r . p y
8
9
10     A N   L I S E   S I N T   T I C A
11     ( P a r s e r )
12     p a r s e r . p y
13
14
15     R E P R E S E N T A   O   I N T E R M   D I A
16     ( A S T   -   r v o r e   d e   S i n t a x e
17       A b s t r a c t a )
18     a s t _ n o d e s . p y
19
20
21     A V A L I A   O
22     ( E v a l u a t o r )
23     e v a l . p y
24
25
26     R e s u l t a d o   ( S a   d a )
27
```

3.2.1 Componentes Principais

Análise Léxica

O módulo `lexer.py` implementa a tokenização do código fonte, identificando:

- Palavras-chave (`let`, `fun`, `if`, `match`, etc.)
- Operadores (`+`, `-`, `*`, `/`, `==`, `!=`, etc.)
- Literais numéricos e booleanos
- Identificadores
- Comentários

Análise Sintática

O módulo `parser.py` utiliza a biblioteca PLY (Python Lex-Yacc) para:

- Processar tokens
- Validar a gramática
- Construir uma Árvore de Sintaxe Abstracta (AST)

Representação Intermédia

O ficheiro `ast_nodes.py` define classes para todas as construções:

- `NumberLiteral`, `BoolLiteral`
- `VarExpr`, `IfExpr`, `MatchExpr`
- `BinOp`, `UnaryOp`, `CallExpr`
- Padrões: `IntPattern`, `WildcardPattern`, `VarPattern`
- Definições: `LetStmt`, `FunDef`

Avaliação

O módulo `eval.py` implementa a interpretação de programas:

- Avaliação recursiva de expressões
- Pattern matching robusto
- Gestão de contexto e variáveis
- Suporte a funções

Capítulo 4

Implementação

4.1 Parte A - Análise Léxica

A primeira etapa focou-se na construção de um analisador léxico robusto.

4.1.1 Funcionalidades

- Extração de tokens do código fonte
- Identificação de palavras reservadas
- Reconhecimento de operadores e delimitadores
- Processamento de comentários de linha (`--`) e bloco (`{- -}`)
- Suporte a literais numéricos e booleanos

4.1.2 Estrutura do Lexer

```
1 # Exemplo de tokeniza o
2 Entrada: let x = 42
3 Tokens:  ['let', 'x', '=', '42']
```

4.2 Parte B - Análise Sintática

A segunda etapa focou-se na construção da gramática e parser.

4.2.1 Gramática

A gramática define a sintaxe válida para a linguagem LFun:

```
1 program : expr
2         | stmt_list
3
4 expr : NUMBER
5      | TRUE | FALSE
6      | ID
7      | expr '+' expr
8      | expr '-' expr
9      | expr '*' expr
10     | expr '/' expr
11     | 'if' expr 'then' expr 'else' expr
12     | 'match' expr 'with' case_list
13     | ID '(' expr ')'
```

4.2.2 Utilização de PLY

O projecto utiliza a biblioteca **PLY** (Python Lex-Yacc) para:

- Definição declarativa da gramática
- Geração automática do parser
- Tratamento de conflitos de ambiguidade
- Recuperação de erros de sintaxe

4.3 Parte C - Avaliação Básica

A terceira etapa integrou o parser com um sistema básico de avaliação.

4.3.1 Características

- Interpretação de expressões aritméticas
- Avaliação de expressões condicionais (if-then-else)
- Processamento de padrões simples
- Interface REPL (Read-Eval-Print Loop)

4.4 Parte D - Representação Intermédia e Pattern Matching

A quarta e final etapa implementou um sistema robusto com AST e pattern matching avançado.

4.4.1 Nós da AST

```
1 class NumberLiteral(Node):
2     def __init__(self, value):
3         self.value = int(value)
4
5 class BoolLiteral(Node):
6     def __init__(self, value):
7         self.value = bool(value)
8
9 class VarExpr(Node):
10     def __init__(self, name):
11         self.name = name
12
13 class BinOp(Node):
14     def __init__(self, op, left, right):
15         self.op = op
16         self.left = left
17         self.right = right
18
19 class MatchExpr(Node):
20     def __init__(self, expr, cases):
21         self.expr = expr
22         self.cases = cases
23
24 class Case(Node):
25     def __init__(self, patterns, expr):
26         self.patterns = patterns
27         self.expr = expr
```

4.4.2 Pattern Matching

O sistema suporta múltiplos tipos de padrões:

- **Padrões Literais:** Correspondência exacta de valores
- **Alternativas:** Múltiplos padrões com o operador `—`
- **Wildcard:** Símbolo `_` para qualquer valor
- **Captura de Variáveis:** Nomes começados com minúscula

4.4.3 Exemplos de Pattern Matching

```
1 # Padrão com alternativas
2 match x with
3 | 1 | 2 | 3 -> "um, dois ou tres"
4 | _ -> "outro valor"
5
6 # Captura de variável
7 match x with
8 | n when n > 0 -> "positivo"
9 | n -> n + 1
10
11 # Padrão literal com wildcard
12 match (x, y) with
13 | (0, _) -> "x zero"
14 | (_, 0) -> "y zero"
15 | _ -> "ambos não zero"
```

4.4.4 Avaliação Genérica

O avaliador opera exclusivamente sobre nós AST:

```
1 def evaluate(node, context):
2     if isinstance(node, NumberLiteral):
3         return node.value
4     elif isinstance(node, BoolLiteral):
5         return node.value
6     elif isinstance(node, VarExpr):
7         return context.get(node.name)
8     elif isinstance(node, BinOp):
9         left = evaluate(node.left, context)
10        right = evaluate(node.right, context)
11        return apply_op(node.op, left, right)
12    elif isinstance(node, MatchExpr):
13        return evaluate_match(node, context)
14    # ... outros casos
```

Capítulo 5

Tecnologias Utilizadas

- **Python 3:** Linguagem de implementação
- **PLY (Python Lex-Yacc):** Framework para lexer e parser
- **Pytest:** Framework para testes

Capítulo 6

Testes e Validação

6.1 Estratégia de Testes

O projecto inclui múltiplos níveis de testes:

6.1.1 Testes Unitários

Testes dos componentes individuais através de `test_eval.py`:

```
1 # Testes b sicos
2 def test_absolute():
3     assert evaluate("absoluto(-5)") == 5
4     assert evaluate("absoluto(5)") == 5
5
6 def test_boolean():
7     assert evaluate("eZero(0)") == True
8     assert evaluate("eZero(1)") == False
9
10 def test_alternatives():
11     assert evaluate("avalia(-1)") == True
12     assert evaluate("avalia(0)") == False
13     assert evaluate("avalia(2)") == True
```

6.1.2 Cobertura de Testes

Os testes cobrem:

- Expressões aritméticas
- Expressões booleanas
- Padrões literais
- Padrões com alternativas
- Wildcard patterns
- Captura de variáveis
- Nested match
- Casos não-exaustivos

6.2 Resultados de Teste

Exemplos de execução bem-sucedida:

```
1 absoluto(-5) = 5
2 absoluto(5) = 5
3 eZero(0) = True
4 eZero(1) = False
5 avalia(-1) = True
6 avalia(0) = False
7 avalia(2) = True
8 sign(-5) = -1
9 sign(0) = 0
10 sign(7) = 1
```

Capítulo 7

Resultados e Demonstração

7.1 Interface REPL

O projeto inclui um REPL (Read-Eval-Print Loop) interativo:

```
1 $ python D/main.py
2 LFun Interpreter 1.0
3 Type 'exit' or 'quit' to exit
4
5 > let absoluto x = match x with | n when n < 0 -> -n | n -> n
6 > absoluto(-10)
7 10
8
9 > match 5 with | 1 | 2 | 3 -> "pequeno" | _ -> "grande"
10 "grande"
```

7.2 Exemplo Completo

Um exemplo demonstrando vários aspetos da linguagem:

```
1 -- Defini o de fun o com pattern matching
2 let sign x = match x with
3 | 0 -> 0
4 | n when n < 0 -> -1
5 | _ -> 1
6
7 -- Utiliza o da fun o
8 sign(-5) -- retorna: -1
9 sign(0) -- retorna: 0
10 sign(7) -- retorna: 1
11
12 -- Pattern matching com alternativas
13 match 2 with
14 | 1 | 2 | 3 -> "um dois ou tr s"
15 | _ -> "outro"
16 -- retorna: "um dois ou tr s"
17
18 -- Express es condicionais
19 if 5 > 3 then "verdadeiro" else "falso"
20 -- retorna: "verdadeiro"
```


7.3 Conformidade com Requisitos

Requisito	Status	Evidência
Pattern Matching (literal, alternativas, wildcard, captura)		test_eval_more.py
Representação Intermedia Genérica		ast_nodes.py
Avaliação sobre IR		eval.py
Suporte a Funções		let, fun, signatures
Testes Abrangentes		test_eval.py, test/
Qualidade de Código		Sem código duplicado

Capítulo 8

Conclusão

Este trabalho prático permitiu consolidar conhecimentos fundamentais sobre processamento de linguagens, desde a análise léxica até à interpretação avançada com representação intermédia.

8.1 Aprendizagens Principais

1. **Separação de Responsabilidades:** Cada etapa (lexer, parser, AST, avaliador) tem um papel bem definido
2. **Representação Intermédia:** Decisão crítica de usar AST em vez de dicionários permite maior flexibilidade
3. **Pattern Matching:** Implementação robusto de múltiplos tipos de padrões
4. **Testes:** Importância de cobertura abrangente para validar comportamento

8.2 Funcionalidades Implementadas

- Análise léxica completa com suporte a comentários
- Parser robusto com tratamento de ambiguidades
- Representação Intermédia genérica (AST)
- Avaliador recursivo com pattern matching avançado
- Suporte a funções e assinaturas de tipo
- REPL interactivo
- Suite de testes abrangente

8.3 Possíveis Melhorias Futuras

- Optimizações de performance (compilação ou caching)
- Geração de código intermediário (bytecode)
- Tratamento de erros mais informativo

8.4 Reflexão Final

O projecto foi realizado com sucesso, cumprindo todos os requisitos especificados. O sistema implementado é robusto, bem-estruturado e facilmente extensível. A arquitectura em camadas permite fáceis mudanças futuras sem afectar outras partes do sistema.

A escolha de utilizar uma AST genérica desde o início mostrou-se acertada, permitindo operações de transformação e análise sofisticadas no futuro.

Apêndice A

Estrutura de Ficheiros

```
1 TP_PL202526/  
2     README.md  
3     pl-tp-enunciado_pub.pdf  
4     A/  
5         lexer.py  
6         lexer_test  
7     B/  
8         lexer.py  
9         grammar.py  
10        parser.py  
11        parsetab.py  
12    C/  
13        lexer.py  
14        parser.py  
15        main.py  
16        parsetab.py  
17        test  
18    D/  
19        ast_nodes.py  
20        eval.py  
21        grammar.py  
22        lexer.py  
23        main.py  
24        README.md  
25        test_eval.py  
26        test_eval_more.py  
27        run_tests.py  
28        test/  
29            test_eval_pytest.py
```

Apêndice B

Repositório GitHub

O código fonte, histórico de versões e documentação complementar deste trabalho estão disponíveis publicamente no GitHub através do seguinte link:

<code>://github.com.PL202526</code>
